

DekantPM — Mathematical Analysis

Status: Canonical math reference for the L_2 -norm AMM with linear probability display and opt-in smooth-kernel settlement (`kernel_width > 0`). Last updated 2026-05-31. For the interactive notebook see [MATH_ANALYSIS_INTERACTIVE.html](#) (or the offline bundle in [math_offline/](#)).

1. Introduction: Continuous Prediction Markets

1.1 What Are Prediction Markets?

A prediction market is a speculative market where participants trade contracts whose payoffs depend on the outcome of future events. In a **binary** prediction market, a contract pays \$1 if event E occurs and \$0 otherwise. The market price of such a contract reflects the crowd's aggregate belief about the probability of E .

1.2 From Discrete to Continuous

Traditional prediction markets handle **discrete** outcomes: "Will candidate A win?" (Yes/No) or "Which team wins the championship?" (Team 1 / Team 2 / ... / Team N). But many real-world quantities are **continuous**: "What will the temperature be on July 1st?" "When will GPT-5 be released?" "What will the price of ETH be on December 31st?"

A **continuous prediction market** elicits a full probability distribution over a continuous outcome space. Rather than asking "Is the answer X?" it asks "What does the crowd believe the probability density function over all possible answers looks like?"

1.3 The Goal

The goal of a continuous prediction market is to:

1. **Aggregate beliefs** — Allow many participants to express nuanced probabilistic views about a continuous variable.
 2. **Maintain a consensus distribution** — The market's state at any time encodes the crowd's aggregate probability distribution.
 3. **Incentivize accuracy** — Traders who move the distribution toward the truth profit; those who move it away lose.
 4. **Provide liquidity** — An automated market maker (AMM) ensures that trades can always be executed without requiring a counterparty.
-

2. Mathematical Framework: The L₂-Norm AMM

2.1 Core Setup

Consider a market with N outcomes (or bins, in the continuous case). We define:

- $\mathbf{k}$ (= `total_minted`): Total collateral deposited into the AMM as complete sets.
- $\mathbf{h} = (\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_n)$ (= `reserves`): The AMM's reserve of outcome tokens for each outcome.
- $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n)$: Positions held collectively by all traders, where:

$$x_i = k - h_i$$

The vector $\mathbf{x}$ represents how many tokens of each outcome have been sold by the AMM to traders.

2.2 The Invariant

The AMM enforces a **constant L₂-norm** on the position vector:

$$\|\mathbf{x}\|_2 = \sqrt{(\sum_i x_i^2)} = k$$

Equivalently, squaring both sides:

$$\sum_{i=1}^N x_i^2 = k^2$$

Or in terms of reserves:

$$\sum_{i=1}^N (k - h_i)^2 = k^2$$

This defines a **hypersphere** of radius k centered at the origin in position space. All valid AMM states lie on this hypersphere.

2.3 Why L₂-Norm? — The Market Scoring Rule Property

Suppose the true probability distribution over outcomes is $\mathbf{p} = (p_1, \dots, p_n)$ with $\sum p_i = 1$. A rational trader maximizes the expected value of their holdings on the invariant surface:

$$\max_{\mathbf{x}} \mathbf{p} \cdot \mathbf{x} \quad \text{subject to} \quad \|\mathbf{x}\|_2 = k$$

By the **Cauchy-Schwarz inequality**, the dot product $\mathbf{p} \cdot \mathbf{x}$ is maximized when $\mathbf{x}$ is proportional to $\mathbf{p}$:

$$\mathbf{x}^* = k \cdot \mathbf{p} / \|\mathbf{p}\|_2$$

where $\|p\|_2 = \sqrt{\sum p_i^2}$. This means **the optimal position vector is directly proportional to the true probability distribution**. A market populated by rational traders will converge to a state where:

$$x_i \propto p_i$$

This makes the L_2 -norm AMM a **market scoring rule** — the relative sizes of positions encode the market's aggregate belief about outcome probabilities.

2.4 Probability Extraction

Given that $x \propto p$ at equilibrium, the displayed probability is the **linear** normalization:

$$\hat{p}_i = x_i / \sum_j x_j = (k - h_i) / \sum_j (k - h_j)$$

This recovers the true probabilities at the Cauchy-Schwarz optimum: if $x^* = k \cdot p / \|p\|_2$, then $x_i / \sum x_j = p_i / \sum p_j = p_i$. The implementation scales probabilities by $S = 10^9$:

$$\hat{p}_{i_scaled} = x_i \cdot S / \sum_j x_j$$

The sum normalization $\sum \hat{p}_i = 1$ is enforced explicitly rather than falling out of the invariant.

2.5 Linear Display: Why Not the Quadratic Form

An alternative "quadratic" extraction $\hat{p}_i = x_i^2 / k^2$ is invariant-derived ($\sum \hat{p}_i = 1$ follows from $\sum x_i^2 = k^2$ without renormalization) and was the original DekantPM display. It was retired because the squaring step introduces a systematic non-linear distortion: traders saw probabilities compressed toward the extremes and could not read the market state at face value.

Example (binary market, true probability $p_1 = 0.7$, $p_2 = 0.3$):

$$\|p\|_2 = \sqrt{(0.49 + 0.09)} = \sqrt{0.58} \approx 0.7616$$

$$\text{Quadratic: } \hat{p}_1 = 0.49/0.58 \approx 0.845 \quad \hat{p}_2 = 0.09/0.58 \approx 0.155$$

$$\text{Linear: } \hat{p}_1 = 0.7 \quad \hat{p}_2 = 0.3$$

A "70% true" outcome rendered as "84.5%" with the quadratic display.

Distortion table the legacy quadratic display would have produced:

True probability	Quadratic $\hat{p}$ (legacy)	Linear $\hat{p}$ (current)
50%	50.0%	50.0%
60%	69.2%	60.0%
70%	84.5%	70.0%
80%	94.1%	80.0%
90%	98.8%	90.0%

The linear display is exact at equilibrium: a "70% displayed" outcome corresponds to a 70% true equilibrium probability. The trade-off is that the renormalization $\sum x_j$ must be computed and tracked, rather than being implied by the invariant. The buy-to-price function, fee calculations, and AMM core all target the **linear** probability — a trader who targets "70%" reaches a state where $x_i / \sum x_j = 0.70$.

Important: Throughout this document, "probability" and $\hat{p}_i$ refer to the **linear** extraction $x_i / \sum x_j$. The quadratic form is mentioned only when contrasting with the prior version.

3. Market Initialization

When a market is created with initial liquidity **L** and **N** outcomes:

1. Set $k = L$ (total_minted = initial liquidity).
2. Compute uniform positions: $x_i = \lfloor \sqrt{(L^2/N)} \rfloor$ for all i (integer square root).
3. Compute reserves: $h_i = L - x_i$ for all i .
4. Set $k^2 = L^2$.
5. Initial LP receives L shares.

The initial linear probability for each outcome is:

$$\hat{p}_i = x_i / \sum_j x_j = (L/\sqrt{N}) / (N \cdot L/\sqrt{N}) = 1/N$$

This gives a uniform prior — every outcome starts equally likely.

Example: Binary market ($N = 2$) with $L = 1,000,000$ (1 USDC):

- $x_1 = x_2 = \lfloor \sqrt{(10^{12}/2)} \rfloor = 707,106$
- $h_1 = h_2 = 1,000,000 - 707,106 = 292,894$
- $\hat{p}_1 = \hat{p}_2 = 707,106 / (2 \cdot 707,106) = 0.5 \checkmark$

4. Discrete Trading

4.1 Buy (Single Outcome)

A trader spends c collateral (after fees) to buy tokens of outcome i .

Algorithm:

1. Record current positions: $x_j = k - h_j$ for all j .
2. Mint complete sets: $h_j \leftarrow h_j + c$ for all j . New total: $k' = k + c$.
3. After minting, positions are unchanged: $x'_j = k' - h'_j = (k+c) - (h_j+c) = k - h_j = x_j$.
4. The invariant now requires $\sum x'^2_j = k'^2$ but we have $\sum x_j^2 = k^2 < k'^2$.
5. The "gap" $k'^2 - k^2$ is filled by increasing x'_i (reducing h_i) for the target outcome:

$$x'_i = \sqrt{(k'^2 - \sum_{j \neq i} x_j^2)}$$

6. Tokens received by trader: $\Delta x = x'_i - x_i$.
7. Update reserve: $h_i = k' - x'_i$.

Price impact: A larger purchase c increases x_i relative to other positions, raising $\hat{p}_i = x_i / \sum x_j$. The square root in step 5 creates a **concave** relationship between position size and collateral — each additional unit costs more, providing natural slippage protection.

4.2 Sell (Single Outcome)

A trader returns t tokens of outcome i to the AMM and receives collateral.

Algorithm:

1. Current position: $x_i = k - h_i$. Require $x_i \geq t$.
2. Return tokens: $h_i \leftarrow h_i + t$ (position decreases: $x'_i = x_i - t$).
3. Compute new k from the invariant:

$$k' = \sqrt{(\sum_j x'^2_j)} \quad \text{where } x'_j = x_j \text{ for } j \neq i, \quad x'_i = x_i - t$$

4. Collateral out: $\Delta c = k - k'$.
5. Burn complete sets: $h_j \leftarrow h_j - \Delta c$ for all j .

Key difference from buy: The sell operation decreases k (total_minted goes down), while buy increases it. No quadratic solving is needed — k' is computed directly via square root.

4.3 Price Impact Analysis

For a buy of amount c on outcome i in a binary market ($N = 2$) with current state (x_1, x_2, k) :

Before trade:

$$\hat{p}_1 = x_1 / (x_1 + x_2)$$

After trade (buying outcome 1):

$$\begin{aligned} k' &= k + c \\ x'_{11} &= \sqrt{(k'^2 - x_2^2)} \\ \hat{p}'_{11} &= x'_{11} / (x'_{11} + x_2) \end{aligned}$$

The price impact increases with c/k (trade size relative to liquidity). For small trades ($c \ll k$), the price impact is approximately linear; for large trades, the square root in x'_{11} creates diminishing returns — each additional unit of collateral buys fewer tokens.

5. Distribution Trading (Continuous Markets)

5.1 Bin Discretization

DekantPM implements continuous markets by **discretizing** the outcome range $[a, b]$ into N equally-spaced bins. Each bin j covers the interval:

$$[a + j \cdot (b-a)/N, \quad a + (j+1) \cdot (b-a)/N)$$

with center:

$$\text{center}_j = a + (2j + 1)(b - a) / (2N), \quad j = 0, 1, \dots, N-1$$

Traders express beliefs as Normal distributions $N(\mu, \sigma^2)$ over this range. The program computes weights for each bin proportional to the Normal PDF evaluated at the bin center.

5.2 Normal PDF Bin Weights

Given parameters μ (mean) and σ (standard deviation):

Step 1 — Compute raw weights. For each bin j , compute the standardized distance:

$$z_j = (\text{center}_j - \mu) / \sigma$$

If $|z_j| > Z_CUTOFF$ ($= 5$), set $w_j = 0$ (tail cutoff). Otherwise:

$$w_j = \exp(-z_j^2/2)$$

Note: this is the Normal PDF kernel (without the $1/\sqrt{2\pi}$ normalizing constant, which cancels during normalization anyway).

Step 2 — On-chain approximation. The exponential is read from a precomputed lookup table covering $|z| \in [0, 5]$ in steps of 0.1, giving 51 entries of $\exp(-z^2/2)$ at SCALE = 10^9 precision. For an arbitrary $|z|$, the on-chain function:

1. Locates the bracketing table indices $\lfloor 10 \cdot |z| \rfloor$ and $\lfloor 10 \cdot |z| \rfloor + 1$.
2. Linearly interpolates between the two table values using the fractional part.

This replaces the earlier degree-4 Taylor polynomial, whose error grew to 10–50% in the tails ($|z| \in [2.5, 5]$) and corrupted the kernel weight vector for broad distributions. The table + linear interpolation keeps the relative error below $\sim 0.5\%$ across the entire $|z| \in [0, 5]$ range while costing only one table lookup, one subtraction, one multiplication, and one division per bin.

All arithmetic uses fixed-point with denominator $S = 10^9$. $Z_CUTOFF = 5$ ensures that bins beyond 5σ from the mean receive zero weight (excluding less than 6×10^{-7} of the probability mass).

Step 3 — Normalize. Weights are scaled so $\sum w_j = S (= 10^9)$:

$$W_j = w_j \cdot S / \sum_m w_m$$

A final correction step adjusts the largest weight to ensure the sum is exactly S , absorbing any rounding remainder.

5.3 Distribution Buy

A trader spends c collateral (after fees) to buy a weighted bundle across all bins.

Algorithm:

1. Compute dot products before minting:
 - $XW = \sum_j x_j \cdot W_j$ (position-weight correlation)
 - $W^2 = \sum_j W_j^2$ (sum of squared weights)
2. Mint complete sets: $h_j \leftarrow h_j + c$ for all j , $k' = k + c$.
3. Solve for λ . We want positions:

$$x'_j = x_j + \lambda \cdot W_j / W^2$$

where λ is a scalar controlling the trade size. The invariant requires $\sum (x'_j)^2 = k'^2$. Expanding:

$$\begin{aligned} \sum (x_j + \lambda W_j / W^2)^2 &= k'^2 \\ \sum x_j^2 + 2(\lambda / W^2) \sum x_j W_j + (\lambda^2 / W^4) \sum W_j^2 &= k'^2 \\ k^2 + 2\lambda \cdot XW / W^2 + \lambda^2 / W^2 &= k'^2 \end{aligned}$$

Multiplying by W^2 :

$$\lambda^2 + 2 \cdot XW \cdot \lambda - W^2 \cdot (k'^2 - k^2) = 0$$

Taking the positive root (see Appendix A for full derivation):

$$\lambda = \sqrt{(XW^2 + W^2 \cdot \text{excess})} - XW \quad \text{where } \text{excess} = k'^2 - k^2$$

4. Tokens out per bin:

$$\text{tokens_out}_j = \lambda \cdot W_j / W^2$$

5. Update reserves: $h_j \leftarrow h_j - \text{tokens_out}_j$.

Interpretation: The parameter λ controls the magnitude of the position shift. The direction W/W^2 is the normalized weight vector. The trade moves the market's distribution toward the trader's expressed Normal distribution, with the shift size determined by how much collateral is spent.

5.4 Distribution Sell

A trader returns **T** total tokens, distributed across bins proportional to weights W .

Algorithm:

1. Distribute tokens: $\text{tokens_for_bin}_j = T \cdot W_j / S$ (since $\sum W_j = S$, this is proportional).
2. Return to reserves: $h_j \leftarrow h_j + \text{tokens_for_bin}_j$.
3. Compute new total from the invariant:

$$k' = \sqrt{\sum_j (k - h'_j)^2}$$

4. Collateral out: $\Delta c = k - k'$.
5. Burn complete sets: $h_j \leftarrow h_j - \Delta c$ for all j .

No quadratic is needed because the tokens being returned are known, and k' is computed directly via square root.

5.5 Buy-to-Price (Price-Targeted Trading)

A trader can specify a **target probability** p^* for outcome i (in the linear sense: $x_i / \sum x_j = p^* / S$), and the AMM computes how much collateral is needed.

Derivation:

We want $\hat{p}_i = p^* / S$ after the trade, meaning $x'_i = (p^* / S) \cdot \sum_j x'_j$. Let $q = p^* / S$ and $\Sigma_{-i} = \sum_{j \neq i} x_j$ (note: the linear formula uses Σ of positions, not Σ of squares).

Since non-target positions don't change during a buy (the complete-set minting preserves them), $x'_j = x_j$ for $j \neq i$. The target relation becomes:

$$\begin{aligned}x'_i &= q \cdot (x'_i + \Sigma_{-i}) \\x'_i (1 - q) &= q \cdot \Sigma_{-i} \\x'_i &= q \cdot \Sigma_{-i} / (1 - q)\end{aligned}$$

The buy mechanics in §4.1 still apply — given the target x'_i we recover k' from the invariant:

$$\begin{aligned}k'^2 &= x'^2_i + \sum_{j \neq i} x_j^2 \quad (\text{squared sum of non-target positions, denote } \Sigma^2_{-i}) \\k' &= \sqrt{x'^2_i + \Sigma^2_{-i}} \\c &= k' - k\end{aligned}$$

So the algorithm is: (a) compute target x'_i from the linear relation, (b) compute k' from the L_2 invariant, (c) return $c = k' - k$.

Sell-to-price works analogously — compute the target position x_{target} from the same linear relation, then derive $k' = \sqrt{x_{\text{target}}^2 + \Sigma^2_{-i}}$ and return $\text{tokens_in} = x_i - x_{\text{target}}$.

6. Liquidity Provision

6.1 Adding Liquidity

An LP deposits d collateral. The AMM scales all reserves proportionally:

$$h'_j = h_j \cdot (k + d) / k \quad \text{for all } j$$

This preserves the probability distribution. To see why: positions scale as $x'_j = k' - h'_j = (k+d) - h_j(k+d)/k = (k+d)(1 - h_j/k) = (k+d) \cdot x_j/k$. Since all positions scale by the same factor $(k+d)/k$, the linear ratio $x_i/\sum x_j$ is invariant. Liquidity increases without shifting the market's view.

LP shares:

- First LP (market creator at initialization): $\text{shares} = L$ (initial liquidity).
- Subsequent LPs: $\text{shares_new} = L_{\text{total}} \cdot d / k$, where L_{total} is current total LP shares and $k = \text{total_minted}$.

After scaling: $k' = k + d$, $k'^2 = (k + d)^2$.

6.2 Removing Liquidity

An LP burns s shares out of L_{total} total shares.

If market is Active or PendingResolution:

$$\text{collateral_out} = k \cdot s / L_{\text{total}}$$

Then scale reserves down proportionally:

$$h'_j = h_j \cdot (k - \text{collateral_out}) / k \quad \text{for all } j$$

This shrinks the market uniformly, preserving the probability distribution but reducing depth.

If market is Resolved:

$$\text{collateral_out} = h_{\text{winning}} \cdot s / L_{\text{total}}$$

where $h_{\text{winning}} = \text{reserves}[\text{resolved_outcome}]$. After resolution, traders have claimed their winning tokens at 1:1. The remaining reserves of the winning outcome belong to LPs. Each LP gets their proportional share of this residual.

Fee share (in both cases):

$$\begin{aligned} \text{fee_share} &= \text{lp_fee_accumulated} \cdot s / L_{\text{total}} \\ \text{total_payout} &= \text{collateral_out} + \text{fee_share} \end{aligned}$$

6.3 LP Economics

LPs effectively take the other side of every trade. They profit from:

1. **Fees** — A share of every trade fee (default: 50% of the 0.3% trade fee = 0.15% per trade).
2. **Mean reversion** — If prices move but revert, LPs capture the round-trip spread.

LPs face risk from:

1. **Divergence loss** — If the market moves permanently to a concentrated distribution, the LP's share of the losing outcomes becomes worthless. In a binary market, if $\hat{p}_1$ goes from 50% to 90%, the LP portfolio is heavily weighted toward the losing outcome. This is analogous to impermanent loss in DEX AMMs.
2. **Informed traders** — Traders with superior information extract value from LPs by buying underpriced outcomes before the market corrects.

Divergence loss formula (binary market): If the market moves from uniform ($\hat{p} = 0.5$) to a final state $\hat{p}_1 = q$, the LP's position value relative to the initial deposit is:

$$\text{value_ratio} = (1 + \sqrt{(1-q)/q}) / 2 \quad (\text{for the case where outcome 1 wins})$$

At $q = 0.5$: ratio = 1 (no loss). At $q = 0.9$: ratio ≈ 0.67 (33% loss if outcome 1 wins).

7. Resolution and Payout

7.1 Discrete Markets

An oracle provides the winning outcome index. All holders of winning outcome tokens redeem them 1:1 for collateral (minus redemption fee):

$$\text{net_payout} = \text{winning_tokens} - \lfloor \text{winning_tokens} \cdot \text{redemption_fee_bps} / 10000 \rfloor$$

Losing outcome tokens are worth zero.

7.2 Continuous Markets

The oracle provides a realized value $v \in [a, b]$. The winning bin is determined by:

$$\text{bin_win} = \lfloor (v - a) \cdot N / (b - a) \rfloor, \quad \text{clamped to } [0, N-1]$$

Continuous markets settle in one of two modes, chosen at market creation via the `kernel_width` parameter W :

- **Winner-take-all ($W = 0$):** Same as discrete markets — only `bin_win`'s holders redeem 1:1 (minus fee). All other bins pay zero. Identical to §7.1.
- **Smooth-kernel settlement ($W \geq 1$):** Adjacent bins also receive partial payouts, smoothing the discontinuity at bin boundaries.

7.2.1 Smooth-Kernel Payout

For a market with kernel width $W > 0$ and resolved bin `bin_win`, each bin i pays a fraction of face value to its holders, weighted by a **triangular kernel** centered at `bin_win`:

$$K(i, \text{bin_win}, W) = \max(0, (W + 1 - |i - \text{bin_win}|) / (W + 1))$$

- $K(\text{bin_win}) = 1$ (full face value).
- $K(\text{bin_win} \pm 1) = W/(W+1)$, $K(\text{bin_win} \pm 2) = (W-1)/(W+1)$, ...
- $K(\text{bin_win} \pm d) = 0$ for $d > W$ (outside the kernel's support).

A trader holding h_i tokens of bin i receives gross payout:

$$\text{gross_payout} = \lfloor \sum_i h_i \cdot K(i, \text{bin_win}, W) \cdot s / S \rfloor$$

where s is a per-market `scaling_factor` ($\in [0, S]$) set at resolution. The scaling factor enforces vault solvency:

$$s = \min(S, k \cdot S / \sum_j K_j \cdot \text{trader_totals}_j)$$

where `trader_totals[j]` is the total tokens of bin j held by all traders (LP-excluded), and $K_j = K(j, \text{bin_win}, W)$. The scaling factor is computed once at resolution and stored on the Market account, so every claimant applies the same divisor.

Why scale? Without it, the sum of kernel-weighted claims can exceed the vault. The scaling factor caps the maximum total payout at exactly k — see the solvency proof in §7.3. When the unscaled claims would already fit ($\sum K_j \cdot \text{trader_totals}_j \leq k$), $s = S$ and the payout is the unscaled kernel sum.

7.2.2 Worked Example (Smooth Kernel)

Market with $N = 5$ bins, $k = 1,000,000$, $W = 1$. Resolved bin = 2.

- $K = [0, 0.5, 1.0, 0.5, 0]$ (only bins 1, 2, 3 inside the kernel's support).
- Suppose trader holdings: $h = [0, 200, 1000, 200, 0]$.
- $\sum K_j \cdot h_j = 0.5 \cdot 200 + 1.0 \cdot 1000 + 0.5 \cdot 200 = 100 + 1000 + 100 = 1200$.
- If `trader_totals` match h (single trader, no LP residual), $s = \min(S, k \cdot S / 1200) = S$ ($1200 \leq 10^6$).
- Gross payout = $\lfloor 1200 \cdot S / S \rfloor = 1200$.

The trader collects 1200 vs. the 1000 they would have received under WTA, because adjacent bins contributed partial credit. Now consider the kernel's support being saturated by aggregate holdings ($\sum K_j \cdot \text{trader_totals}_j = 5,000,000$ with $k = 1,000,000$): $s = S/5 = 0.2 \cdot S$, so every claim is scaled by 0.2 and total payouts equal exactly k .

7.3 Solvency Guarantee

The system is always solvent. The proof is by cases on the settlement mode.

Claim: The vault holds at least k collateral at all times, and the maximum possible redemption at resolution is at most k .

Shared setup (both modes):

1. **Complete-set minting:** Every token in circulation was created as part of a complete set. For every unit of outcome token i held by traders (x_i), there are $h_i = k - x_i$ units in the AMM reserves. Total tokens per outcome: $x_i + h_i = k$.
2. **Vault balance:** Every complete-set mint deposits exactly 1 unit of collateral per token set. After minting k total sets: $\text{vault} \geq k$ (plus accumulated fees, minus LP withdrawals — but LP withdrawals are bounded by their proportional share of k).

Case 1 — WTA (binary, multi-outcome, continuous with $W = 0$):

Only the winning outcome redeems. The maximum number of winning tokens held by all traders is $x_{\text{winning}} \leq k$ (since $x_{\text{winning}}^2 \leq \sum x_j^2 = k^2$). Therefore total redemption $\leq k \leq$ vault balance. ■

Case 2 — Smooth-kernel continuous ($W \geq 1$):

The aggregate claim across all traders is:

$$\begin{aligned} \text{total_claim} &= \sum_i \text{trader_totals}_i \cdot K(i, \text{bin_win}, W) \cdot s / S \\ &= (s / S) \cdot \sum_i K_i \cdot \text{trader_totals}_i \end{aligned}$$

By construction $s = \min(S, k \cdot S / \sum_j K_j \cdot \text{trader_totals}_j)$. Substituting:

$$\begin{aligned} \text{total_claim} &\leq (S / S) \cdot k = k \quad \text{when the constraint is active } (\sum_j K_j \cdot t_j > k) \\ \text{total_claim} &= \sum_i K_i \cdot \text{trader_totals}_i \leq k \quad \text{when } s = S \text{ (constraint inactive)} \end{aligned}$$

Either way, $\text{total_claim} \leq k \leq \text{vault balance}$. ■

LP residual: After all traders claim:

- WTA: LPs receive $h_winning = k - x_winning$ (the AMM's unclaimed share of the winning outcome).
- Kernel: LPs receive the remaining vault balance, $k - \text{total_claim}$, distributed proportionally to LP shares.

8. Worked Example: Continuous Market Lifecycle

This section walks through a complete continuous market from creation to resolution with concrete numbers. We use a simplified 5-bin market for clarity.

8.1 Market Creation

Market: "ETH price on March 31, 2026"

- Range: [2000, 4000] (USDC, scaled by 10^9 on-chain)
- Bins: $N = 5$ (bin width = 400 USDC)
- Initial liquidity: $L = 1,000,000$ (1 USDC in micro-units)

Bin structure:

Bin	Range	Center
0	[2000, 2400)	2200
1	[2400, 2800)	2600
2	[2800, 3200)	3000
3	[3200, 3600)	3400
4	[3600, 4000]	3800

Initialization:

```

k = 1,000,000
xi =  $\lfloor \sqrt{(10^{12}/5)} \rfloor = \lfloor \sqrt{(200,000,000,000)} \rfloor = 447,213$  for all i
hi = 1,000,000 - 447,213 = 552,787 for all i
k2 = 1012

```

Initial probabilities: $\hat{p}_i = 447,213 / (5 \times 447,213) = 0.200$ for all i (uniform 20% each). ✓

8.2 Trade 1 — Discrete Buy on Bin 2

Trader Alice believes ETH will be in [2800, 3200). She buys bin 2 with 100,000 collateral (0.1 USDC).

Fee calculation (trade_fee = 30 bps, lp_share = 50%):

```

total_fee =  $\lfloor 100,000 \times 30 / 10,000 \rfloor = 300$ 
lp_fee =  $\lfloor 300 \times 5000 / 10,000 \rfloor = 150$ 
protocol_fee = 300 - 150 = 150
net_collateral = 100,000 - 300 = 99,700

```

Buy computation:

```

k' = 1,000,000 + 99,700 = 1,099,700
k'2 = 1,209,340,090,000

Σ-2 = 4 × 447,2132 = 4 × 199,999,387,369 = 799,997,549,476

x'2 =  $\lfloor \sqrt{(1,209,340,090,000 - 799,997,549,476)} \rfloor = \lfloor \sqrt{(409,342,540,524)} \rfloor = 639,798$ 

tokens_out = 639,798 - 447,213 = 192,585
h2 = 1,099,700 - 639,798 = 459,902

```

Other reserves (after complete-set mint): $h_j = 552,787 + 99,700 = 652,487$ for $j \neq 2$.

New probabilities (linear: $\hat{p}_i = x_i / \sum x_j$ where $\sum x_j = 639,798 + 4 \cdot 447,213 = 2,428,650$):

```

p̂2 = 639,798 / 2,428,650 ≈ 0.263 (was 0.200, rose to ~26%)
p̂j = 447,213 / 2,428,650 ≈ 0.184 for j ≠ 2 (was 0.200, fell to ~18.4%)

Check: 0.263 + 4 × 0.184 = 0.263 + 0.736 = 0.999 ≈ 1 ✓

```

Note: the linear display rises more gently than the legacy quadratic would have. Under the quadratic formula the same trade would have shown $\hat{p}_2 \approx 0.339$ — visually exciting but a distortion. Linear is the actual marginal price.

Alice's portfolio: 192,585 tokens of bin 2. Cost: 100,000 collateral (including fees).

8.3 Trade 2 — Distribution Buy

Trader Bob believes ETH will be around \$3,000 with moderate uncertainty. He expresses this as $N(\mu = 3000, \sigma = 400)$ and spends 200,000 collateral (0.2 USDC).

Step 1: Compute bin weights.

Bin j	Center	$z_j = (\text{center} - 3000)/400$	$\exp(-z_j^2/2)$	Normalized $W_j (\times 10^9)$
0	2200	-2.0	0.1353	54,511,000
1	2600	-1.0	0.6065	244,262,000
2	3000	0.0	1.0000	402,619,000
3	3400	+1.0	0.6065	244,262,000
4	3800	+2.0	0.1353	54,346,000

(Weights adjusted so $\sum W_j = 10^9$ exactly.)

Step 2: Fee calculation.

```
total_fee = [200,000 × 30 / 10,000] = 600
net_collateral = 200,000 - 600 = 199,400
```

Step 3: Pre-mint dot products (using state after Trade 1).

Current positions: $x_0 = x_1 = x_3 = x_4 = 447,213$ and $x_2 = 639,798$.

```
XW = x0W0 + x1W1 + x2W2 + x3W3 + x4W4
    ≈ 447,213 × 54.5M + 447,213 × 244.3M + 639,798 × 402.6M
    + 447,213 × 244.3M + 447,213 × 54.3M
    ≈ 524,080 × 109 (approximate)

W2 = W02 + W12 + W22 + W32 + W42
    ≈ (54.5M)2 + (244.3M)2 + (402.6M)2 + (244.3M)2 + (54.3M)2
    ≈ 2.84 × 1017 (approximate)
```

Step 4: Mint + solve quadratic.

```
k' = 1,099,700 + 199,400 = 1,299,100
excess = k'2 - k2 = 1,299,1002 - 1,099,7002 ≈ 478,000 × 106

λ = √(XW2 + W2·excess) - XW
```

Step 5: Tokens out per bin — $\text{tokens_out}_j = \lambda \cdot W_j / W^2$.

The distribution buy gives more tokens for the central bins (especially bin 2) and fewer for the tails. After the trade, the probability distribution is more concentrated around \$3,000.

Post-trade probability distribution (approximate, linear):

Bin	Before Trade 2	After Trade 2
0	18.4%	~13%
1	18.4%	~20%
2	26.3%	~34%
3	18.4%	~20%
4	18.4%	~13%

The market now reflects a bell-shaped consensus centered on bin 2 (\$2800-\$3200).

8.4 Adding Liquidity

LP Carol adds 500,000 collateral (0.5 USDC) to deepen the market.

```
k_before = 1,299,100
k_after = 1,299,100 + 500,000 = 1,799,100
scale_factor = 1,799,100 / 1,299,100 ≈ 1.3849

h'_j = h_j × 1,799,100 / 1,299,100    for all j

LP shares for Carol = L_total × 500,000 / 1,299,100
```

Key property: All probabilities remain unchanged after adding liquidity. The market's view doesn't shift — only the depth increases. Subsequent trades will have lower price impact because k is larger.

8.5 Resolution

The market expires. The oracle reports: **ETH price = \$3,150** ($= 3,150 \times 10^9$ on-chain).

Bin determination:

```
bin = [(3150 - 2000) × 5 / (4000 - 2000)] = [1150 × 5 / 2000] = [2.875] = 2
```

Winning bin: 2 (covering [\$2800, \$3200)).

Payouts:

- **Alice** holds 192,585 tokens of bin 2.
 - Gross payout: 192,585 collateral units (1:1 redemption).
 - Redemption fee (50 bps): $\lfloor 192,585 \times 50 / 10,000 \rfloor = 962$.
 - Net payout: 191,623.
 - **Profit:** $191,623 - 100,000 = \mathbf{91,623}$ (91.6% return).

- **Bob** holds tokens across all bins from his distribution buy. Only bin 2 tokens redeem.
 - His bin 2 tokens (from the distribution buy) redeem 1:1 minus fee.
 - His tokens in bins 0, 1, 3, 4 are worth zero.
 - His profit depends on how many bin 2 tokens he received vs. total cost.
- **Carol** (LP) claims her proportional share of the remaining reserves of bin 2:

$$\text{lp_payout} = h_2_final \times \text{Carol_shares} / L_total + \text{fee_share}$$

Key observation: The resolution maps a continuous value (\$3,150) to a discrete bin (bin 2). Any value in [\$2,800, \$3,200) would have resolved identically. This is the fundamental discretization trade-off — see Section 10.

8.6 Lifecycle Summary

```

Creation:    Uniform distribution (20% each bin), k = 1,000,000
  ↓
Trade 1:     Alice buys bin 2 → probability shifts to 34%
  ↓
Trade 2:     Bob buys N(3000, 400) → bell curve centered on bin 2 (~40%)
  ↓
Add LP:      Carol adds liquidity → probabilities unchanged, depth increases
  ↓
Deadline:    Market transitions to PendingResolution
  ↓
Resolution:  Oracle reports $3,150 → bin 2 wins → 1:1 redemption
  ↓
Claims:      Alice profits, Bob partially profits, Carol gets LP residual + fees

```

9. Fee Structure

9.1 Trade Fees

On every trade with gross amount G:

```

total_fee = [G · trade_fee_bps / 10000]
lp_fee = [total_fee · lp_fee_share_bps / 10000]
protocol_fee = total_fee - lp_fee
net_amount = G - total_fee

```

Default parameters: trade_fee = 30 bps (0.3%), lp_share = 5000 bps (50% of fee).

9.2 Redemption Fee

On payout claims:

```
redemption_fee = [gross_payout * redemption_fee_bps / 10000]  
net_payout = gross_payout - redemption_fee
```

Default: 50 bps (0.5%). The redemption fee accrues to the protocol treasury.

9.3 Fee Impact on Trading

Fees reduce the effective collateral entering the AMM. For a buy with gross amount G :

```
effective_collateral = G - [G * fee_bps / 10000]
```

The trader pays G but only $G - \text{fee}$ enters the AMM engine. This means:

- The actual price impact is slightly less than a fee-free trade of size G .
- The trader receives fewer tokens per unit of gross collateral.
- A round-trip (buy then sell) always costs at least $2 \times \text{fee_rate}$, creating a natural bid-ask spread.

9.4 Fee Accounting

Fees are split between two accumulators on the market account:

- **lp_fee_accumulated** (u128): LP share, distributed proportionally when LPs withdraw.
- **protocol_fee_accumulated** (u64): Protocol share, swept to treasury via a separate `collect_fees` instruction.

10. The Discretization Approach

10.1 How It Works

DekantPM approximates continuous outcome spaces by dividing the range $[a, b]$ into N equally-spaced bins ($N \in [2, 256]$). Each bin is treated as a discrete outcome. Traders express continuous beliefs (Normal distributions) that are projected onto bin weights.

This is a **finite-dimensional approximation** of the infinite-dimensional L_2 function space described in theoretical models (see Section 12).

10.2 Advantages

1. **Simplicity:** The same L_2 -norm AMM engine handles both discrete and continuous markets. No separate continuous math is needed at the AMM level.

2. **On-chain efficiency:** The AMM operates on fixed-length vectors (max 256 elements). All operations are $O(N)$ — computing sums, updating reserves, solving quadratics — feasible within Solana's compute budget.
3. **Composability:** Discrete outcome tokens for each bin can be individually held, transferred, and settled. This enables secondary markets and complex positions.
4. **Exact solvency:** The complete-set minting model guarantees solvency (Section 7.3). Each bin is a separate outcome token with a clear 1:1 redemption path.
5. **No backing constraint complexity:** Unlike the true continuous model, which requires managing a separate backing amount b and solvency constraint $\max(f) \leq b$, the discrete model's solvency is automatic.

10.3 Disadvantages

1. **Resolution granularity:** The market resolves to a single winning bin. Two values in the same bin are treated identically, even if they are different. A market with range $[0, 1000]$ and 100 bins has a resolution of 10 units — outcomes at 105 and 114 both resolve to the same bin.
2. **Distribution approximation error:** The Normal PDF is approximated by a step function (histogram). For small N or large σ relative to the bin width, this is adequate. For narrow σ relative to bin width, significant probability mass may be misallocated between bins.
3. **Fixed bin structure:** Bins are equally spaced. In many applications (e.g., financial prices), a logarithmic or adaptive spacing would better match the distribution of likely outcomes.
4. **Taylor polynomial error:** The degree-4 Taylor approximation of $\exp(-z^2/2)$ degrades for $|z| > 1.5$ (see Section 5.2). For distributions centered near the range edges or very narrow distributions, this introduces systematic bias in the weight vector.
5. **Restricted distribution family:** Only Normal distributions can be traded via the distribution trading interface. Traders cannot express skewed, bimodal, or fat-tailed beliefs directly. (They can, however, place discrete bets on individual bins to construct arbitrary distributions manually.)

11. Limitations and Alternative Approaches

11.1 Limitation: Single-Bin Resolution (RESOLVED for continuous markets)

Historical problem: When a continuous market resolved with $W = 0$ (winner-take-all), the oracle provided a single value v that mapped to exactly one winning bin. All bins except the winner paid zero. This created a discontinuity: two values very close to a bin boundary produced completely different payouts.

Example: Range [0, 100], 10 bins (each width 10). A trader long bin 3 (covers [30, 40)). If the outcome was 39.99 they won everything; if it was 40.01 they won nothing.

Resolution: smooth-kernel settlement. Market creators now opt into smooth-kernel settlement by setting `kernel_width = W ≥ 1` at creation (see §7.2.1). The triangular kernel $K(i, \text{bin_win}, W) = \max(0, (W+1-|i-\text{bin_win}|)/(W+1))$ distributes payout to bins within W steps of the winner. The per-market `scaling_factor`, computed at resolution as $s = \min(S, k \cdot S / \sum K_j \cdot \text{trader_totals}_j)$, guarantees that the sum of kernel-weighted claims never exceeds the vault — preserving the same intrinsic solvency property that WTA enjoys (§7.3, case 2).

Binary and multi-outcome markets still use WTA ($W = 0$ is enforced); the kernel is only meaningful when adjacent bins represent close-but-not-equal outcomes.

11.2 Limitation: Fixed Number of Bins

The Problem: N is set at market creation and cannot change. If a market starts broad (e.g., "ETH price in 2025: \$0-\$100,000" with 100 bins $\rightarrow$ \$1,000/bin) and consensus narrows to a range of \$2,000-\$3,000, most bins are wasted while the interesting range has only 1 bin.

Alternative — Adaptive binning / hierarchical markets. Markets could dynamically refine bins in high-density regions. This is complex on-chain but could be implemented via linked child markets that subdivide a parent bin into finer resolution.

Alternative — The true continuous approach (see Section 12) eliminates bins entirely.

11.3 Limitation: Normal-Only Distributions

The Problem: Traders can only express $\text{Normal}(\mu, \sigma)$ beliefs. Real-world distributions are often skewed (log-normal for prices), heavy-tailed, or multimodal.

Alternative — Support additional distribution families. The bin-weight computation could be extended to log-normal, uniform, or mixture distributions. Each would require its own on-chain weight computation function, increasing program complexity but broadening expressiveness.

Alternative — Allow arbitrary weight vectors. Let traders submit custom weight vectors directly (subject to normalization $\sum W_j = S$). This is maximally flexible but makes the UI much harder — users would need to specify N -dimensional vectors.

11.4 Limitation: Taylor Polynomial Precision (RESOLVED)

Historical problem: A degree-4 Taylor approximation of $\exp(-t/2)$ had relative error up to ~50% for $|z| \in [2.5, 5]$, propagating into the Gaussian bin-weight vector for broad distributions.

Resolution: lookup table + linear interpolation. The current implementation precomputes $\exp(-z^2/2)$ at 51 grid points over $|z| \in [0, 5]$ (step 0.1) and linearly interpolates between them at evaluation time. Relative error is bounded by ~0.5% across the entire support, at the cost of one table lookup, one subtraction, one multiplication, and one division per bin — much cheaper than the higher-degree polynomials originally floated as alternatives.

11.5 Limitation: Compute Budget with Many Bins

The Problem: Distribution buy/sell iterates over all N bins to compute XW , W^2 , and update reserves. For $N = 256$, this is ~ 768 arithmetic operations, which approaches Solana's compute unit limits for a single instruction.

Alternative — Batch processing across multiple transactions. Or use a more efficient AMM that operates on distribution parameters directly (see Section 12).

11.6 Limitation: Quadratic Probability Distortion (RESOLVED)

Historical problem: The original pricing formula $\hat{p}_i = x_i^2/k^2$ created a non-linear distortion between displayed probabilities and true equilibrium probabilities (see §2.5). Traders saw values compressed toward the extremes — a 70% true probability was displayed as 84.5%.

Resolution: linear probability display. DekantPM now computes and displays $\hat{p}_i = x_i/\sum x_j$ throughout the frontend, backend, and contract. The AMM core (invariant, complete-set minting, distribution math) is unchanged because the quadratic formula was only ever a *display* layer — the underlying L_2 AMM mechanics are independent of the choice of probability normalization. The buy-to-price interface (§5.5) was updated to target the linear probability.

12. Comparison with Paradigm's Distribution Markets

12.1 Overview of Paradigm's Approach

The Paradigm paper ("Distribution Markets," Dave White, December 2024) proposes a continuous prediction market operating in **infinite-dimensional function space (L^2)**. The key ideas:

1. **Outcome function tokens:** Instead of discrete outcome tokens, traders hold *functions* f : $\mathbb{R} \rightarrow \mathbb{R}_+$ where $f(x)$ is the payout if outcome x occurs.
2. **Minting:** A constant function $f(x) = b$ can be minted for b dollars (analogous to complete sets in the discrete case).
3. **AMM invariant:** $\|f\|_2 = \sqrt{\int f(x)^2 dx} = k$, a constant L^2 norm.
4. **Solvency:** Requires $\max(f) \leq b$ (the backing amount), which is a separate constraint from the L^2 norm.
5. **Optimality:** By Cauchy-Schwarz in L^2 , the equilibrium position $f^* \propto p$ (the true PDF).

12.2 Structural Comparison

Aspect	DekantPM (Discretized)	Paradigm (True Continuous)
Outcome space	Finite bins: $N \in [2, 256]$	Infinite: $\mathbb{R}$ or any continuous space
Position representation	Vector $x \in \mathbb{R}^N$	Function $f: \mathbb{R} \rightarrow \mathbb{R}$
Invariant	$\sum x_i^2 = k^2$ (finite sum)	$\int f(x)^2 dx = k^2$ (integral)
Pricing	$\hat{p}_i = x_i / \sum x_j$ (linear, per bin)	$\hat{p}(x) \propto f(x)$ (continuous density)
Solvency	Automatic (WTA) or scaled (kernel); always $\leq k$	Requires explicit constraint $\max(f) \leq b$
Backing vs. norm	Same ($k = b$)	Separate ($k \leq b$ typically)
Distribution trading	Normal PDF $\rightarrow$ bin weights $\rightarrow$ weighted discrete trade	Direct trade in distribution parameters (μ, σ)
Collateral model	Complete-set minting: cost = $k' - k$	Max-loss: $-\min_x \{g(x) - f(x)\}$ (can require numerical computation)
Resolution	WTA ($W = 0$) or smooth triangular kernel ($W \geq 1$) with scaling_factor cap	Payout = $f(v)$ where v is realized value (smooth)
LP provision	Proportional scaling of reserves vector	Proportional scaling of position function
On-chain feasibility	Yes (Solana, $O(N)$ per trade)	Requires specialization to specific distribution families

12.3 The Backing Constraint Difference

This is the most fundamental mathematical difference between the two approaches.

DekantPM: In finite dimensions, the L_2 -norm constraint $\|x\|_2 = k$ automatically ensures that no single x_i exceeds k . Since $x_i^2 \leq \sum x_j^2 = k^2$, we have $|x_i| \leq k$. Combined with the 1:1 redemption model, the AMM always has enough reserves to pay any single winning outcome.

Paradigm: In infinite dimensions, $\|f\|_2 = k$ does NOT bound $\max(f)$. Consider $f(x) = M \cdot \delta_\varepsilon(x)$ (a very narrow, very tall spike). Its L^2 norm can be k while its maximum is arbitrarily large. Hence the separate constraint $\max(f) \leq b$.

This means Paradigm's model needs:

- A backing amount $b \geq \max(f)$ to ensure solvency.
- For Normal distributions: a minimum spread $\sigma \geq k^2/(b^2\sqrt{\pi})$ to stay within the backing constraint.
- Numerical computation of max-loss for collateralization.

DekantPM's discretization **sidesteps this entirely** — the finite-dimensional solvency guarantee is intrinsic to the mathematical structure.

12.4 Resolution Model Difference

Paradigm: At resolution with realized value v , a trader holding function f receives $f(v)$. This is a **smooth payout** — nearby outcomes give similar payoffs. If f is a Gaussian centered near v , the payout degrades smoothly as v moves away from the center.

DekantPM: Continuous markets choose one of two modes at creation via `kernel_width`:

- **WTA ($W = 0$):** Bin j wins if v falls in bin j 's range. Payout is 1:1 for winning tokens, 0 elsewhere — a **step function**.
- **Smooth kernel ($W \geq 1$):** Bins within W steps of the winning bin pay a triangular-weighted fraction $K(i, \text{bin_win}, W) \cdot s / S$ per token, with $s = \min(S, k \cdot S / \sum K_j \cdot \text{trader_totals}_j)$ capping aggregate claims at the vault. A trader who bet on "around \$3,000" with a tight distribution gets a high payout if the outcome is \$3,050 (peak bin or one step off) and a lower (but non-zero) payout if it lands W bins away. The chosen W gives the creator a knob between sharp-incentive WTA ($W = 0$) and Paradigm-style smoothing (large W approximating the kernel's triangular envelope).

The discrete kernel converges to Paradigm's continuous payout as $N \rightarrow \infty$ with W scaled appropriately.

12.5 Computational Efficiency

Paradigm (Normal case): A trade changes (μ, σ) to (μ', σ') . The AMM needs to:

1. Verify the L^2 norm constraint (closed-form for Gaussians: $\|f\|_{L^2}^2 = \int f^2 dx$, which has analytic solutions for Gaussian f).
2. Compute max-loss collateral (requires finding min of difference of two Gaussians — generally requires numerical optimization, no closed form).
3. Verify the solvency constraint $\max(f) \leq b$.

This is $O(1)$ in the number of "bins" (there are none), but the numerical max-loss computation adds complexity.

DekantPM: A distribution trade requires $O(N)$ operations to compute weights, XW , W^2 , and update all N reserves. For $N = 256$, this is ~ 1000 arithmetic operations — feasible on Solana within a single transaction.

12.6 Where DekantPM Aligns with Paradigm

1. **Same invariant family:** Both use the L_2 norm as the AMM invariant. DekantPM's $\sum x_i^2 = k^2$ is exactly the discrete analog of Paradigm's $\int f^2 = k^2$.
2. **Same optimality result:** Both invoke Cauchy-Schwarz to show that the equilibrium position is proportional to the true probability distribution.
3. **Same LP model:** Both scale positions proportionally for liquidity provision.
4. **Same market scoring rule property:** In both models, the AMM state encodes the crowd's belief about the probability distribution.

5. **DekantPM converges to Paradigm:** As $N \rightarrow \infty$ and bin width $\rightarrow 0$, DekantPM's discretized model converges to Paradigm's continuous model. The discrete sums become integrals, bin weights become continuous densities, and the step-function payout approaches a smooth payout.

12.7 Where DekantPM Diverges from Paradigm

1. **No separate backing constraint:** DekantPM does not need $b \neq k$. This simplifies the model but means it cannot represent arbitrarily peaked distributions (the bin resolution limits this naturally).
 2. **Discretized smooth payout:** Paradigm's model pays $f(v)$ continuously; DekantPM pays per bin with an opt-in triangular kernel ($W \geq 1$) that approximates smoothness up to the bin resolution.
 3. **Restricted to Normal distributions:** Paradigm discusses Normal, uniform, and even mixtures. DekantPM's distribution trading only implements Normal.
 4. **Collateral model:** Paradigm requires max-loss collateral (the worst-case difference between new and old positions). DekantPM uses complete-set minting — simpler but potentially less capital-efficient (the trader always deposits the full collateral amount, even if their max loss is lower).
-

13. Summary of Mathematical Properties

13.1 Correctness Properties

1. **Invariant preservation:** Every trade (buy, sell, distribution buy/sell) preserves $\sum x_i^2 = k^2$ within tolerance $\varepsilon = 256$. The tolerance accounts for accumulated integer rounding across sqrt and division operations.
2. **Solvency:** The vault always holds $\geq k$ collateral. Maximum redemption at resolution $\leq k$ under both WTA and smooth-kernel settlement (proven in Section 7.3). In kernel mode the `scaling_factor` enforces this bound at claim time.
3. **No arbitrage (within fees):** The displayed linear probabilities sum to 1 (within rounding). There is no risk-free profit loop of buying and selling that nets positive collateral after fees.
4. **Market scoring rule:** At equilibrium, the position vector is proportional to the true distribution, so the linear probabilities $\hat{p}_i = x_i / \sum x_j$ recover the true frequentist probabilities exactly (Cauchy-Schwarz optimum).

13.2 Numerical Properties

1. **Fixed-point arithmetic:** All probability computations use $\text{SCALE} = 10^9$ denominator. This provides ~ 9 decimal digits of precision for probabilities.

2. **Integer square root:** Newton's method converges in ≤ 64 iterations for u128, giving exact $\lfloor \sqrt{n} \rfloor$.
3. **Invariant tolerance:** $\varepsilon = 256$ accounts for accumulated rounding across operations. For typical market sizes ($k \sim 10^6$ - 10^{12}), this is negligible (relative error $< 10^{-7}$).
4. **Overflow safety:** All arithmetic uses checked operations on u128. Maximum $k^2 = (2^{64})^2 = 2^{128}$ — fits in u128. The probability computation $x_i^2 \cdot \text{SCALE}$ can overflow u128 for very large positions; the code handles this gracefully by returning 0 for such overflow cases.

13.3 Key Constants

Constant	Value	Meaning
SCALE	10^9	Fixed-point denominator for probabilities and scaling_factor
INVARIANT_TOLERANCE	256	Maximum allowed invariant drift
MAX_OUTCOMES	32	Maximum discrete outcomes
MAX_BINS	256	Maximum continuous bins
MAX_KERNEL_WIDTH	(per market, $\leq N - 1$)	Triangular kernel half-support; 0 = WTA
Z_CUTOFF	5	Gaussian tail cutoff (5σ)
GAUSSIAN_TABLE_SIZE	51	$\exp(-z^2/2)$ lookup entries over $ z \in [0, 5]$
MIN_LIQUIDITY	10^6	1 USDC minimum initial liquidity
MIN_TRADE_AMOUNT	10^3	0.001 USDC minimum trade
DEFAULT_TRADE_FEE	30 bps	0.3% per trade
DEFAULT_LP_FEE_SHARE	5000 bps	50% of trade fee to LPs
DEFAULT_REDEMPTION_FEE	50 bps	0.5% on payout claims
DEFAULT_CREATION_FEE	50 bps	0.5% on market creation

Appendix A: Derivation of the Distribution Buy Quadratic

We seek to allocate collateral c across bins proportional to weights W , while maintaining the invariant.

After minting complete sets, the new total is $k' = k + c$. We want to find positions:

$$x'_j = x_j + \lambda \cdot W_j / W^2$$

where λ is a scalar and $W^2 = \sum W_j^2$. The invariant requires:

$$\sum (x'_j)^2 = k'^2$$

Expanding:

$$\begin{aligned}\sum (x_j + \lambda W_j / W^2)^2 &= k'^2 \\ \sum x_j^2 + 2(\lambda / W^2) \sum x_j W_j + (\lambda^2 / W^4) \sum W_j^2 &= k'^2 \\ k^2 + 2\lambda \cdot XW / W^2 + \lambda^2 / W^2 &= k'^2\end{aligned}$$

Multiplying by W^2 :

$$\begin{aligned}\lambda^2 + 2 \cdot XW \cdot \lambda + W^2(k^2 - k'^2) &= 0 \\ \lambda^2 + 2 \cdot XW \cdot \lambda - W^2 \cdot \text{excess} &= 0 \quad \text{where excess} = k'^2 - k^2\end{aligned}$$

By the quadratic formula:

$$\begin{aligned}\lambda &= (-2 \cdot XW \pm \sqrt{(4 \cdot XW^2 + 4 \cdot W^2 \cdot \text{excess})}) / 2 \\ &= -XW \pm \sqrt{XW^2 + W^2 \cdot \text{excess}}\end{aligned}$$

Taking the positive root ($\lambda > 0$ for a buy):

$$\lambda = \sqrt{XW^2 + W^2 \cdot \text{excess}} - XW$$

Appendix B: Integer Square Root via Newton's Method

Given $n \in \mathbb{N}$, find $\lfloor \sqrt{n} \rfloor$.

Algorithm:

```
x0 = n/2 + 1
xt+1 = ⌊(xt + n/xt)/2⌋
Stop when xt+1 ≥ xt; return xt.
```

Correctness: Newton's method for $f(x) = x^2 - n$ converges quadratically. The sequence $\{x_t\}$ is monotonically decreasing (after the first step) and bounded below by $\lfloor \sqrt{n} \rfloor$. The stopping condition $x_{t+1} \geq x_t$ detects convergence.

Complexity: For u128 inputs, convergence takes at most 64 iterations (one bit of precision per iteration in the worst case).

Appendix C: Glossary

Symbol	Meaning
N	Number of outcomes or bins
k	Total minted collateral (= total_minted)
h_i	AMM reserve for outcome i
x_i	Position for outcome i: $x_i = k - h_i$
$\hat{p}_i$	Displayed probability of outcome i (linear: $x_i/\sum x_j$)
S	Scale factor (10^9)
W_j	Normalized Gaussian bin weight for bin j ($\sum W_j = S$)
W (kernel)	Kernel width for smooth-kernel settlement; $W = 0 \Rightarrow$ WTA
$K(i, \text{bin_win}, W)$	Triangular settlement kernel: $\max(0, (W+1 - i - \text{bin_win})/(W+1))$
s	Per-resolution <code>scaling_factor</code> $\in [0, S]$ enforcing kernel solvency
trader_totals _j	Sum of bin j holdings across all traders (LP-excluded)
μ	Mean of trader's Normal distribution
σ	Standard deviation of trader's Normal distribution
XW	$\sum x_j W_j$ (position-weight dot product, for distribution buy)
W^2	$\sum W_j^2$ (sum of squared Gaussian weights — distinct from kernel W)
λ	Quadratic solution parameter for distribution buy
L	LP shares total
b	Backing amount (Paradigm model only)
f	Outcome function (Paradigm model only)